Computer Graphics Using OpenGL
INM376 / IN3005

Module Leader – Dr Eddie Edwards

Lecture 06: Lighting

Roadmap for the second half

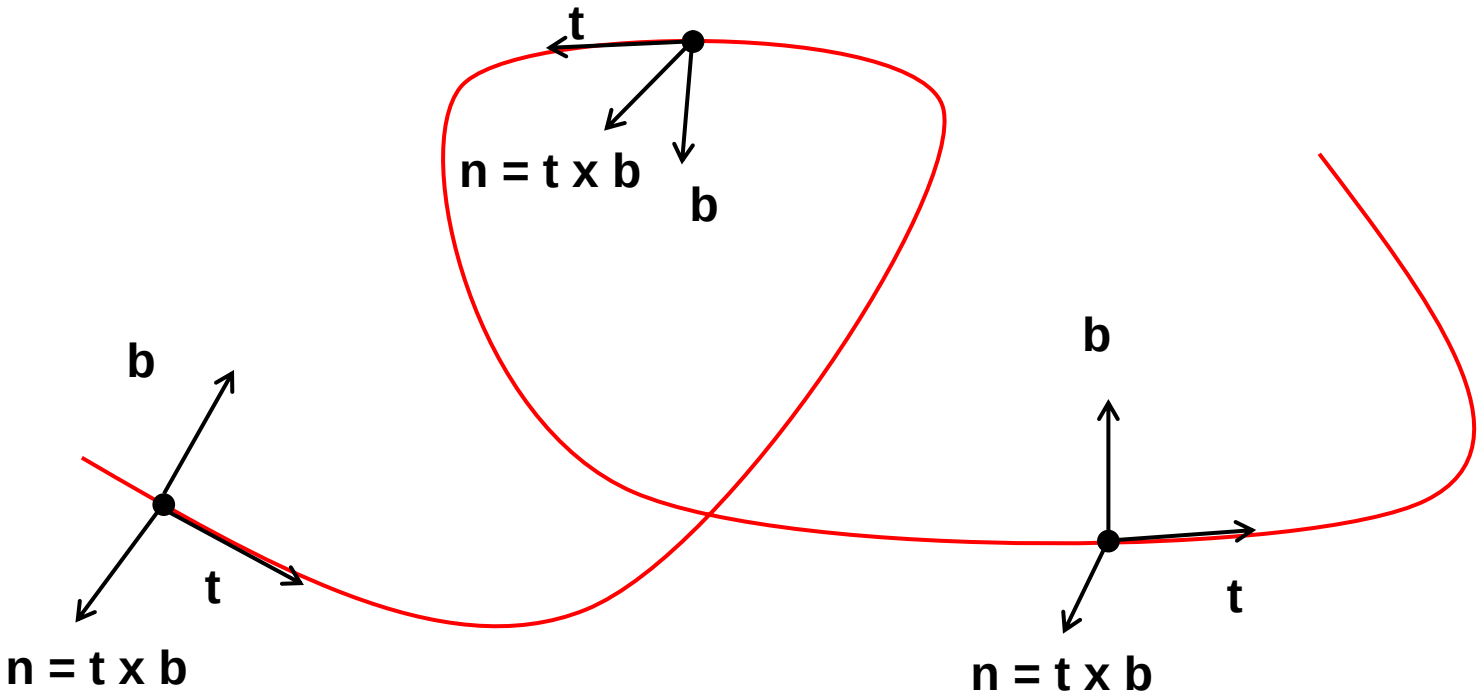
- Session 6 (today)
  - Lecture: Lighting
  - Tutorial: Lab 6 on lighting
- Session 7 (17th March)
  - Lecture: Screenspace techniques
  - Tutorial: Coursework and *informal feedback on your work*
- Session 8 (24th March)
  - Lecture: Intermediate / advanced graphics 1
  - Tutorial: Coursework and *informal feedback on your work*
- Session 9 (7th April)
  - Lecture: Intermediate / advanced graphics 2
  - Tutorial: Coursework: *last session for feedback on coursework*
- **12th April Demo Coursework code submission**
  - 14th April 6-9pm – **Demos!!**
  - **Coursework presentations are mandatory!**
  - Please attend and vote for each others presentations
  - Prizes for the best courseworks as chosen by you!
- **10th May – Final coursework submission (code and report)**

Recap from last time

- Surfaces and mesh representations
- Vertex normals
- Texture mapping – used to apply an image to a surface, kind of like wallpapering
- Mipmapping and anisotropic filtering
- 2D rendering and text

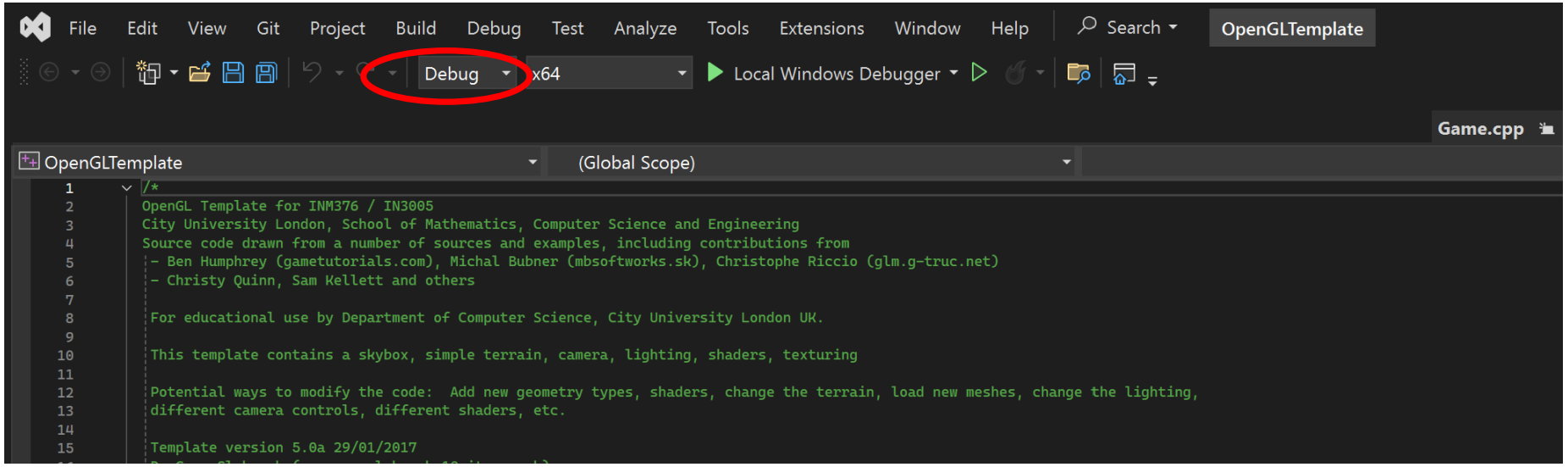
Better Control of Catmull-Rom Spline

- Question: How can I have more control of the track / camera to create a loop?
- Answer: The Catmull-Rom class provided also allows you to specify control *normals* in addition to control points. The control normals can be interpolated like control points in the Sample method to give you **b** vectors.
- At any point along the track, one can fully characterise the **tnb** frame, as you have the **t** vector, the **b** vector is interpolated, and **n** = **t** x **b**.



Debugging of code

- Question: My code isn't working! What do I do?
- Answer 1: *Think* about the code you have written. Why might it fail to produce the right result? Modify code as needed.
- Answer 2: *Debug* it. Visual Studio is a powerful IDE with professional debugging tools.
  - Make sure your program is in Debug mode



  - Set breakpoints (F9) to stop execution at specific places
  - Run the program. When it breaks,
    - Look at values in variables
    - Use F10 and F11 to step over / step into code
    - Use F5 to resume the program

Questions

- Question: How do I debug a shader?

- Answer: Debugging a shader is a bit complicated because it's running on the GPU. Typically there's no easy communication back to the CPU from the GPU. Shaders don't provide a "print" command to output debugging information to a console.

- However, there are some options:
  - In the fragment shader, set the fragment to a specific colour when a bug occurs (a shader version of "print")
  - Use a debugging tool like NVidia Nsight https://developer.nvidia.com/nvidia-nsight-visual-studio-edition
  - Shadertoy to try out shader code online - https://www.shadertoy.com/
  - Several other options:
    https://www.khronos.org/opengl/wiki/Debugging_Tools

Coursework Questions

- The coursework marking sheet is on Moodle. The marking sheet matches the coursework description.
- Use this as a checklist to make sure everything is implemented.
- In general, increased complexity will receive higher marks.
- Note there are separate marking schemes for UG and PG students.

⇨ Advice: *Backup* your coursework as you go in case the files get corrupted or accidentally deleted.

Overview of today's lecture

- Heightmaps
- Lighting
  - Principles
  - Reflectance models
  - Light and material properties
  - Implementation in GLSL

Exercise

- Your colleague is trying to texture map a ground plane with a mipmapped, repeating texture in s and t. However, it's not working as expected!
  1. What is wrong with the following code? *Hint: there may be multiple errors.*

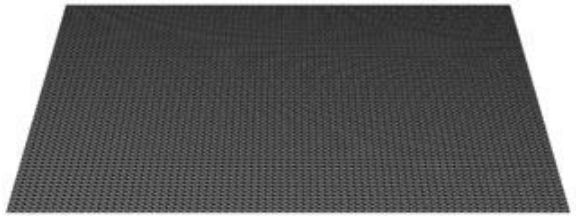
```
m_tTexture.SetSamplerObjectParameter(GL_TEXTURE_MIN_FILTER, GL_LINEAR);
m_tTexture.SetSamplerObjectParameter(GL_TEXTURE_MAG_FILTER, GL_LINEAR_MIPMAP_LINEAR);
m_tTexture.SetSamplerObjectParameter(GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
m_tTexture.SetSamplerObjectParameter(GL_TEXTURE_WRAP_T, GL_REPEAT);
```

  - The minification filter should be `GL_LINEAR_MIPMAP_LINEAR`
  - The magnification filter should be `GL_LINEAR`
  - `GL_REPEAT` should be used for the s wrapping mode

  2. What other things might your colleague want to check?
  - The texture is loaded
  - When the texture is loaded, it is mipmapped
  - Texture coordinates are correct
  - Texture coordinates are read in vertex shader and passed to fragment shader
  - When rendering, texture is bound
  - Sampler in fragment shader is using the correct texture unit
  - In fragment shader, texture is being used (e.g., `bUseTexture = true` in template)

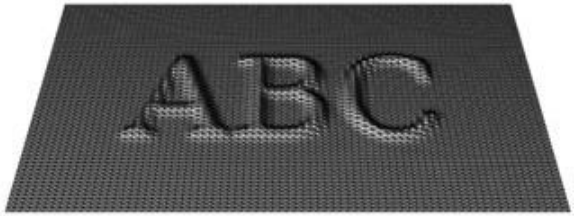
Heightmaps

- A common method of modeling terrain is the use of a *heightmap*.
- A heightmap provides a function indicating a height value for each position along a plane. In our graphics template, this function would have the form y = f(x, z).
- The heightmap usually is stored as a 2D image, where the colour of each pixel represents a displacement of a point on a plane.



original mesh

image

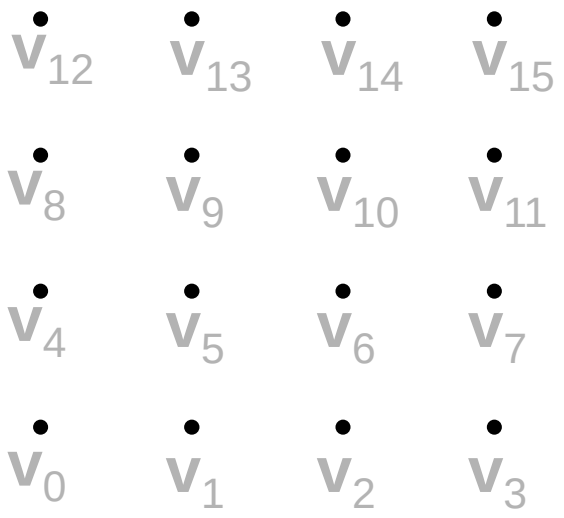


displaced mesh

Forming the mesh

- Read in image representing height field
- Interpret each pixel as a height value $y = f(x, z)$
- Determine position each $[x, y, z]^T$ point in 3D space

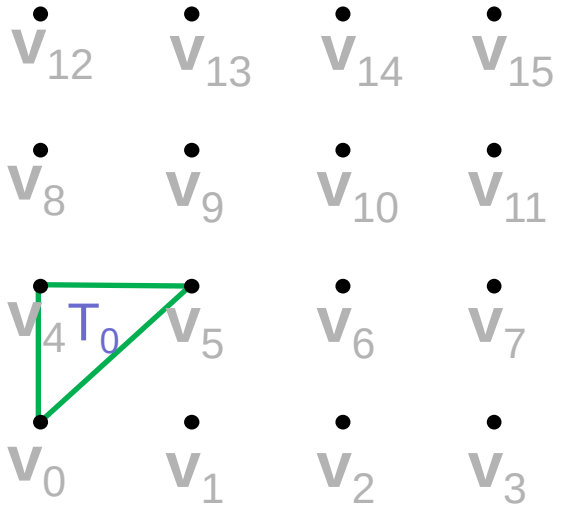
	Vertex list
$\mathbf{v}_0$	x_0,y_0,z_0
$\mathbf{v}_1$	x_1,y_1,z_1
$\mathbf{v}_2$	x_2,y_2,z_2
$\mathbf{v}_3$	x_3,y_3,z_3
$\mathbf{v}_4$	x_4,y_4,z_4
$\mathbf{v}_5$	x_5,y_5,z_5
	...



Forming the mesh

- Read in image representing height field
- Interpret each pixel as a height value y = f(x, z)
- Determine position each $[x, y, z]^T$ point in 3D space
- Construct triangles from grid of points

	Vertex list
$\mathbf{v}_0$	x_0,y_0,z_0
$\mathbf{v}_1$	x_1,y_1,z_1
$\mathbf{v}_2$	x_2,y_2,z_2
$\mathbf{v}_3$	x_3,y_3,z_3
$\mathbf{v}_4$	x_4,y_4,z_4
$\mathbf{v}_5$	x_5,y_5,z_5
	...

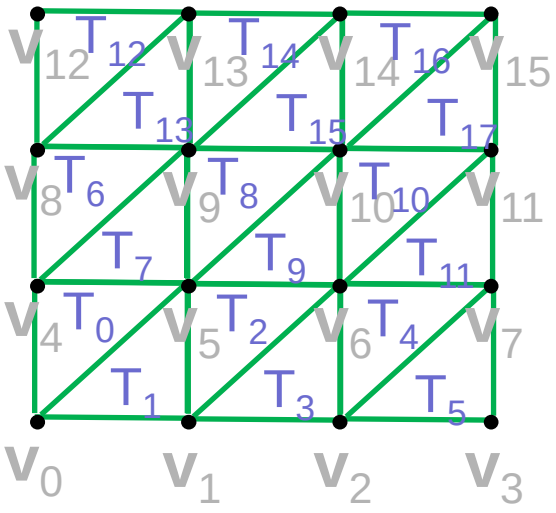


	Triangle list
T_0	**0, 5, 4**

Forming the mesh

- Read in image representing height field
- Interpret each pixel as a height value y = f(x, z)
- Determine position each $[x, y, z]^T$ point in 3D space
- Construct triangles from grid of points

	Vertex list
$\mathbf{v}_0$	x_0,y_0,z_0
$\mathbf{v}_1$	x_1,y_1,z_1
$\mathbf{v}_2$	x_2,y_2,z_2
$\mathbf{v}_3$	x_3,y_3,z_3
$\mathbf{v}_4$	x_4,y_4,z_4
$\mathbf{v}_5$	x_5,y_5,z_5
	...

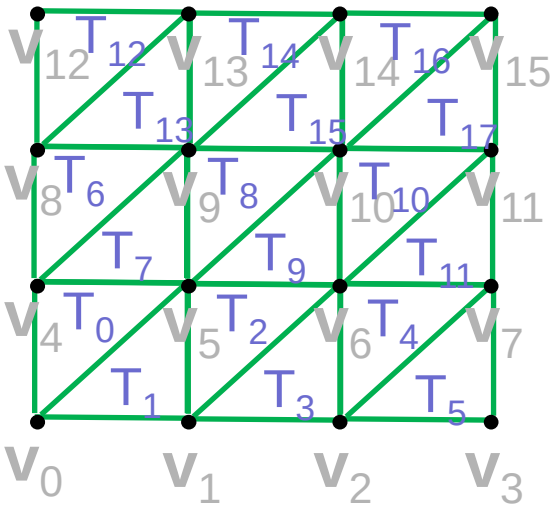


	Triangle list
T_0	**0, 5, 4**
T_1	**0, 1, 5**
T_2	**1, 6, 5**
T_3	**1, 2, 6**
T_4	**2, 7, 6**
	…

Forming the mesh

- Read in image representing height field
- Interpret each pixel as a height value y = f(x, z)
- Determine position each $[x, y, z]^T$ point in 3D space
- Construct triangles from grid of points
- Form face vertex mesh

	Vertex list	On triangle
v_0	x_0,y_0,z_0	0, 1
v_1	x_1,y_1,z_1	1, 3, 2
v_2	x_2,y_2,z_2	3, 5, 4
v_3	x_3,y_3,z_3	5
v_4	x_4,y_4,z_4	0, 7, 6
v_5	x_5,y_5,z_5	0, 1, 2, 9, 8, 7
	...	...



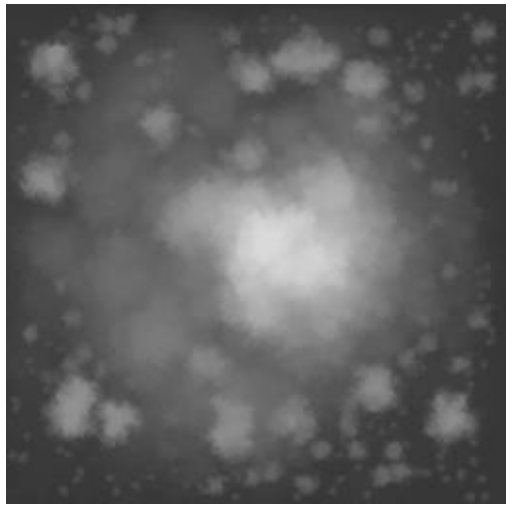
	Triangle list
T_0	0, 5, 4
T_1	0, 1, 5
T_2	1, 6, 5
T_3	1, 2, 6
T_4	2, 7, 6
	…

Keeping track of the height

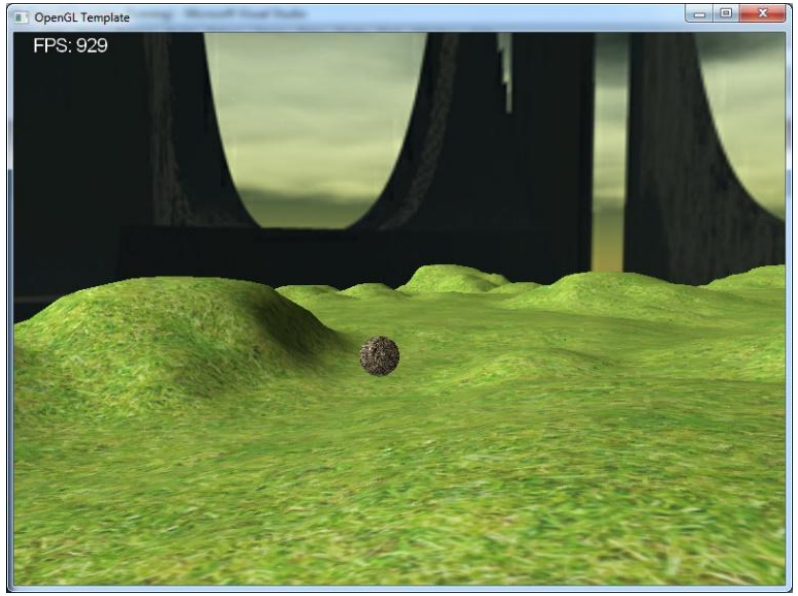
- It is useful to have a function that can return the height y of a point on the plane specified by (x, z).
  - Setting objects so that they are placed on the ground
  - Setting the camera so that it is always y units above the ground
- One way to do this is to map between world coordinates [x, y, z] and the heightmap image.

```
glm::vec3 MHeightMapTerrain::WorldToImageCoordinates(glm::vec3 p) glm::vec3
MHeightMapTerrain::ImageToWorldCoordinates(glm::vec3 p)
```

- Example provided on Moodle:



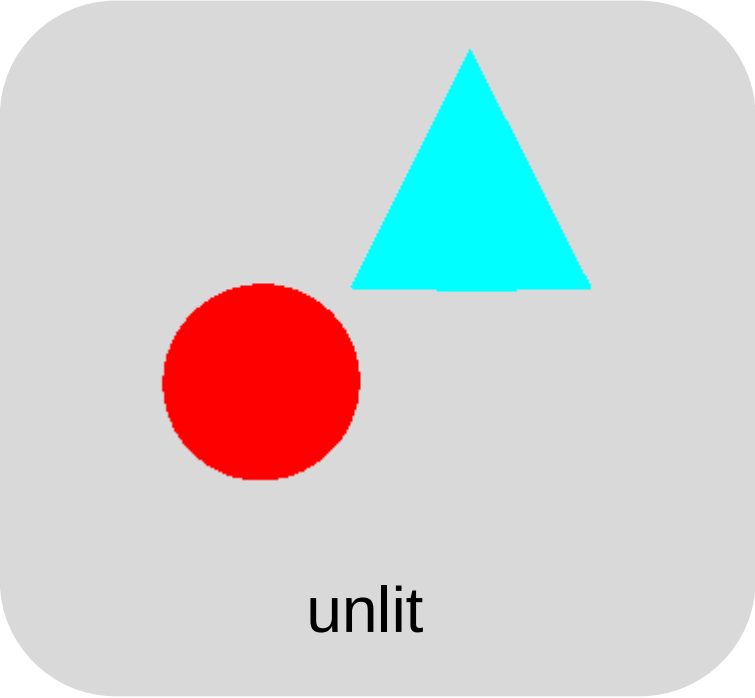
Heightmap image

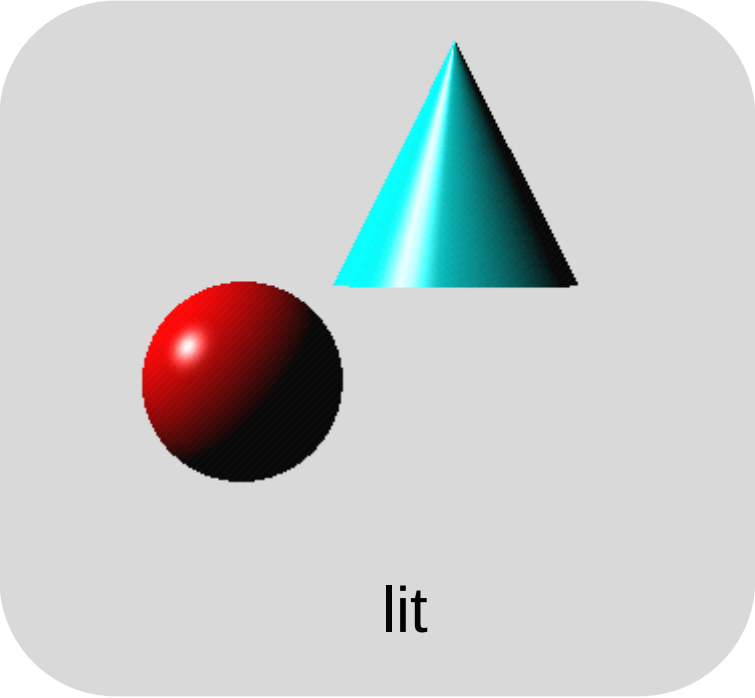


Terrain

Lighting

- Lighting can have a dramatic effect on appearance
- For example, consider the sphere and cone below





Lighting principles

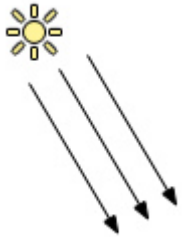
- The colours we see in the real world are based on the interaction between the object materials and the lighting in the scene.
- Using OpenGL, the programmer can mimic this process using mathematical models for lighting and materials. The result is *shading*, that is, varying levels of brightness and darkness that depict depth.
- In the real world, shading results from a multitude of complex interactions, including multiple reflections resulting from interactions amongst surfaces.

Local lighting model in OpenGL

- To do physically-correct shading, one must perform a *global* calculation to see how light bounces (possibly multiple times) off of surfaces. This would include
  - All the light sources
  - All the objects and their materials
- Real lighting requires that objects be lit not only by light sources but by objects themselves – both the same object and other objects. These operations tend to be recursive and computationally expensive.
- In real-time computer games, speed is of the utmost importance.
- Instead, OpenGL processes each primitive independently, or *locally*. As a consequence, objects are lit only by light sources – not by light from other objects.
- This is a highly simplified model compared the complexity of the real world. However, it is suitable for hardware implementation and produces compelling shading for games.

Directional light source

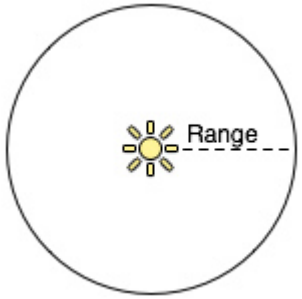
- A *directional light source* is global and considered to be infinitely far away (e.g., the sun). Rays are considered parallel by the time they reach the object.

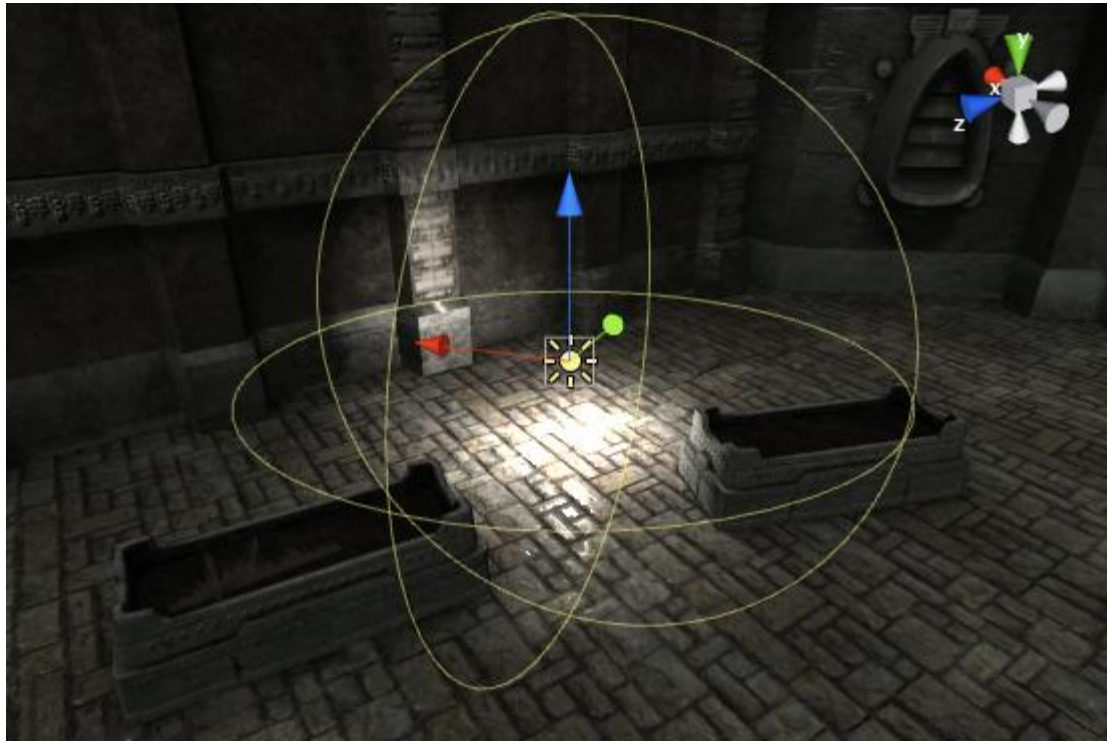


http://unity3d.com/support/documentation/Components/class-Light.html

Point light source

- A *point light source* has a defined point in space, emitting light in all directions (e.g., a spherical street light).
- In the image below, the light is *attenuated* with distance from the light source.

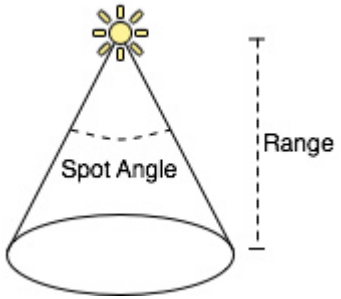


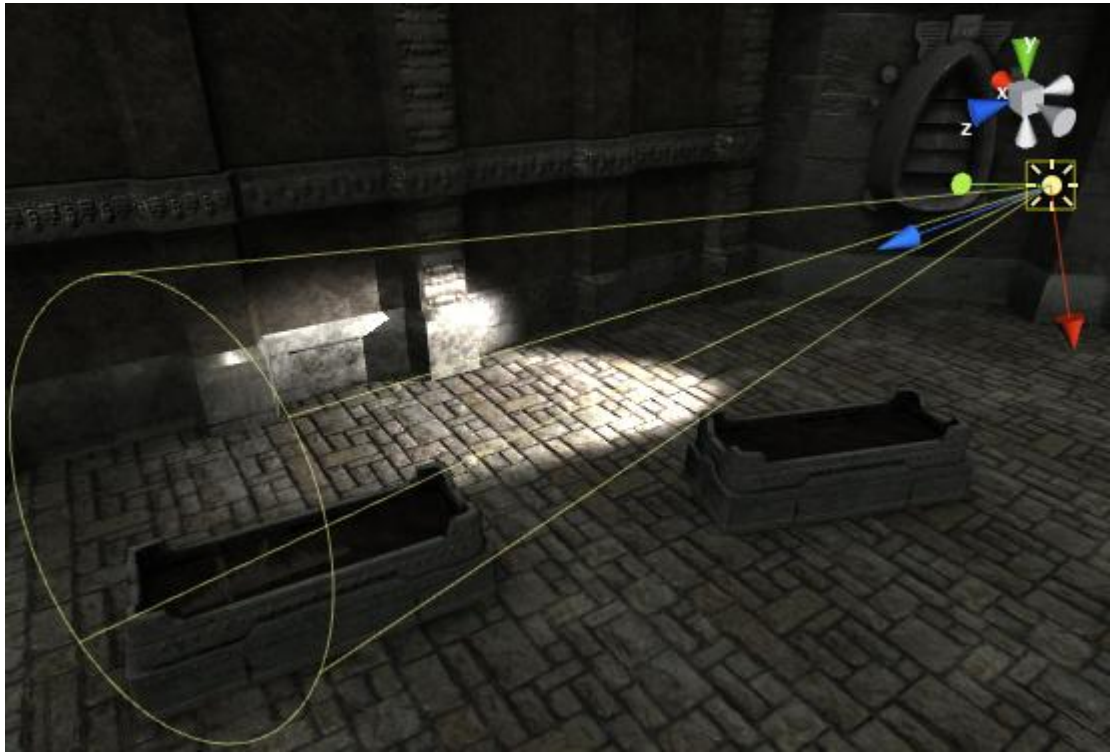


http://unity3d.com/support/documentation/Components/class-Light.html

Spotlight

- A *spotlight* similar to a point light source, but emits light in a specific direction in the shape of a cone.

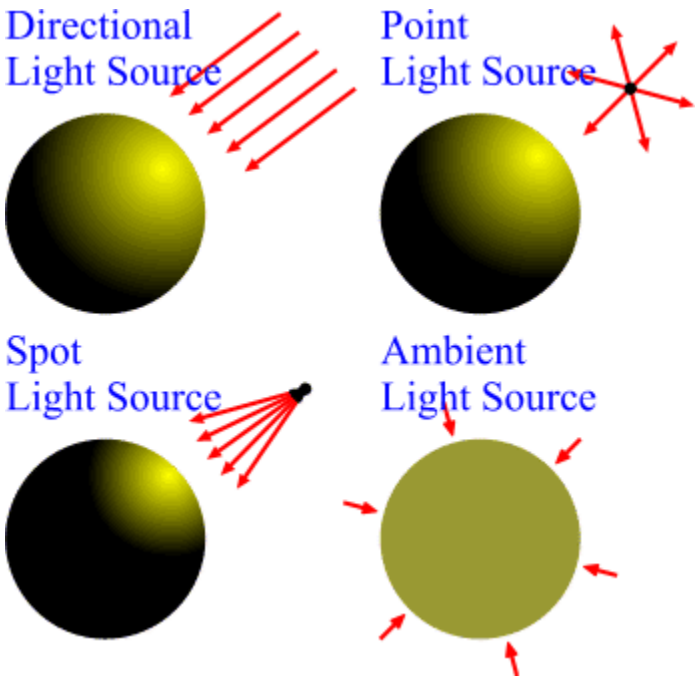




http://unity3d.com/support/documentation/Components/class-Light.html

Ambient light

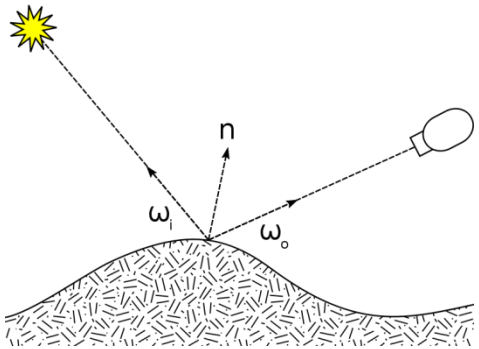
- Approximates the low level of light that is normally present everywhere in a scene (scattered by many objects before reaching the eye).
- It is used to account for all the complex ways in which light can reach an object that are not otherwise addressed by light sources mentioned earlier.

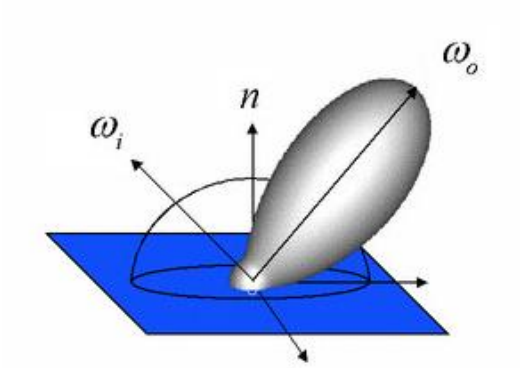


http://www.computing.northampton.ac.uk/~gary/csy3019/CSY3019SectionD.html

Reflectance model

- A reflectance model is a mathematical description of how incoming light is reflected from a surface.
- A fairly general form is the *bidirectional reflectance distribution function* (BRDF)
  - The BRDF is a four-dimensional function that models the amount of light reflected at an opaque surface point. The BRDF returns the ratio of reflected radiance exiting along a direction ω_o to the irradiance incident on the surface from direction ω_i.
  - Note that each direction ω is itself parameterized by azimuth angle ϕ and zenith angle θ.
- This 4D model is rather complex for a real-time computer game. We would prefer a simpler model, and numerous have been proposed in the field of computer graphics.

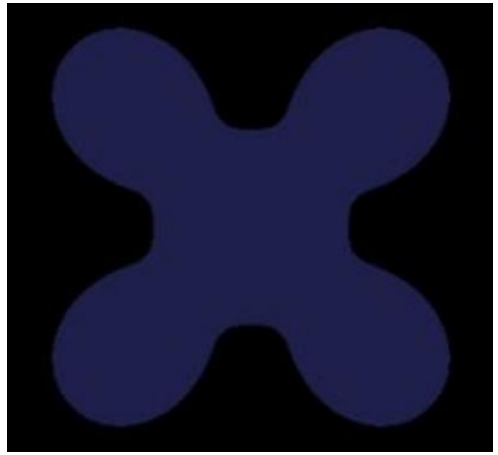




http://www.dlib.org/dlib/february02/goesele/02goesele.html

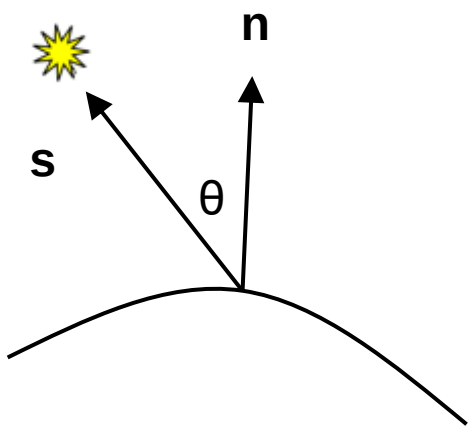
Ambient reflectance

- **Ambient reflection:** A light's ambient component $\mathbf{l}_a$ is reflected by a surface. The amount of light reflection is based on the surface *ambient material*, $\mathbf{m}_a$.
- The colour resulting from the ambient reflection is $\mathbf{c}_a = \mathbf{l}_a$.* $\mathbf{m}_a$, where .* indicates an element-wise product. For example, consider the case below where we have low intensity white ambient light and a blue surface.



Diffuse reflectance

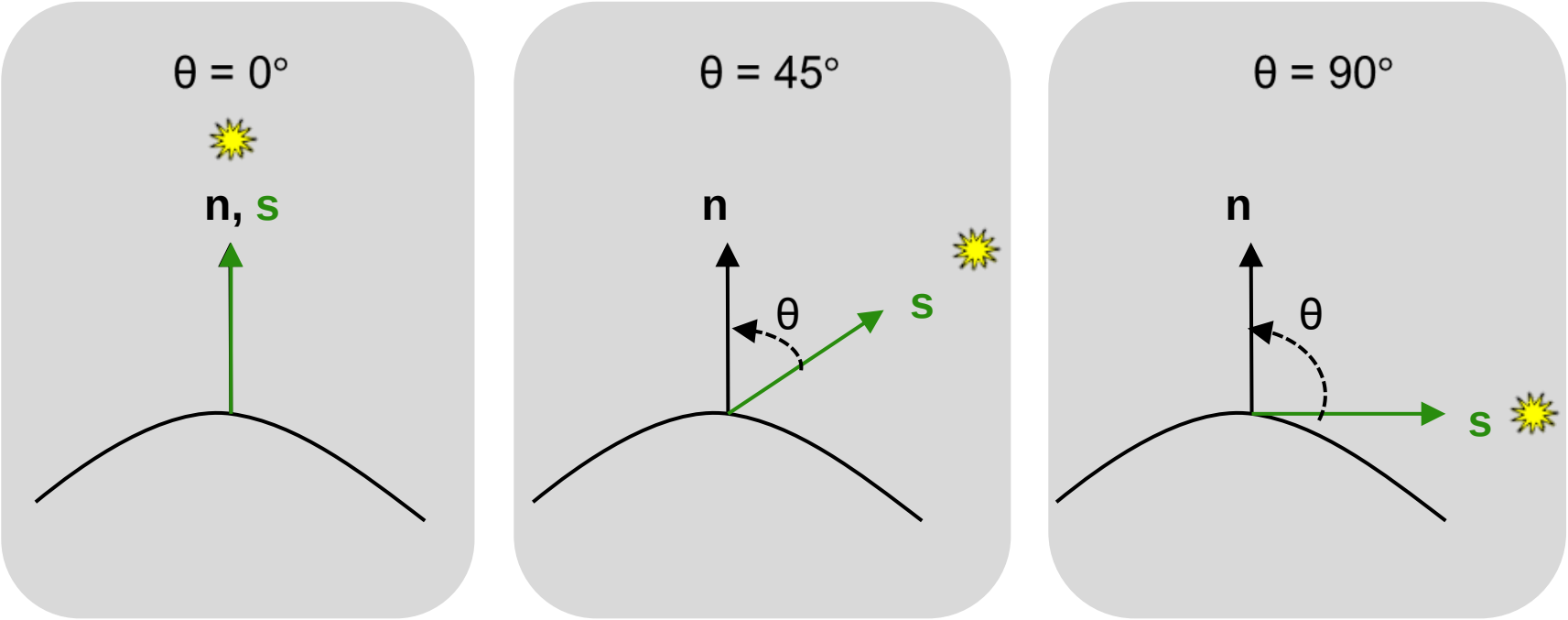
- Although an object illuminated by ambient light is lit in proportion to the ambient intensity, it is uniformly illuminated across the surface.
- Dull or matte surfaces, like chalk, exhibit ***diffuse* reflection** (also known as *Lambertian* reflection).
- These surfaces appear equally bright from all *viewing* angles.
- However, the brightness changes based on the angle between the surface normal ***n*** and the direction to the light source, ***s***. This is conveniently expressed as a dot product where ***n*** and ***s*** are normalised.



$$\mathbf{n} \bullet \mathbf{s} = ||\mathbf{n}||\ ||\mathbf{s}||\ \cos(\theta)$$

n • s

- Normally, all vectors used in lighting are normalised
- So **n • s =** ||**n**|| ||**s**|| cos(θ) = cos(θ)
- Let's consider a few cases

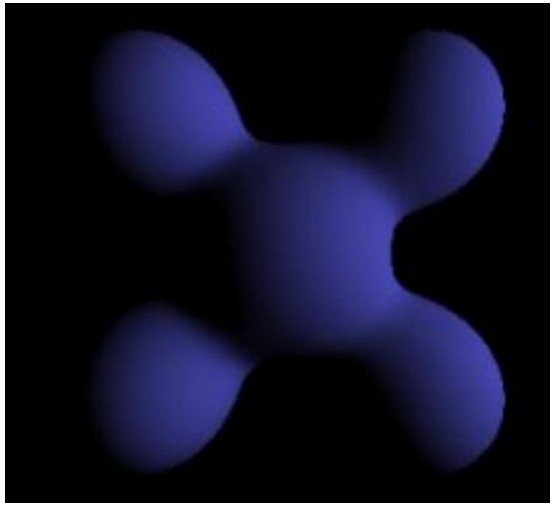


n • s = 1 **n • s** = 0.7071 **n • s** = 0

- If **n • s** < 0, no diffuse reflection is modelled

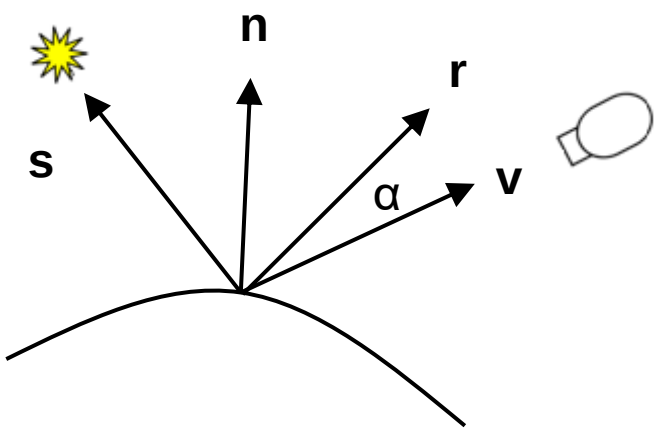
Diffuse term

- A light's diffuse component $\mathbf{l}_d$ is reflected by a surface. The amount of light reflection at each vertex is based on surface normal **n**, the light source direction ***s***, and the surface *diffuse material*, $\mathbf{m}_d$.
- The colour resulting from the diffuse reflection is $\mathbf{c}_d = (\mathbf{l}_d \,.*\, \mathbf{m}_d)\ \mathbf{n} \bullet \mathbf{s}$



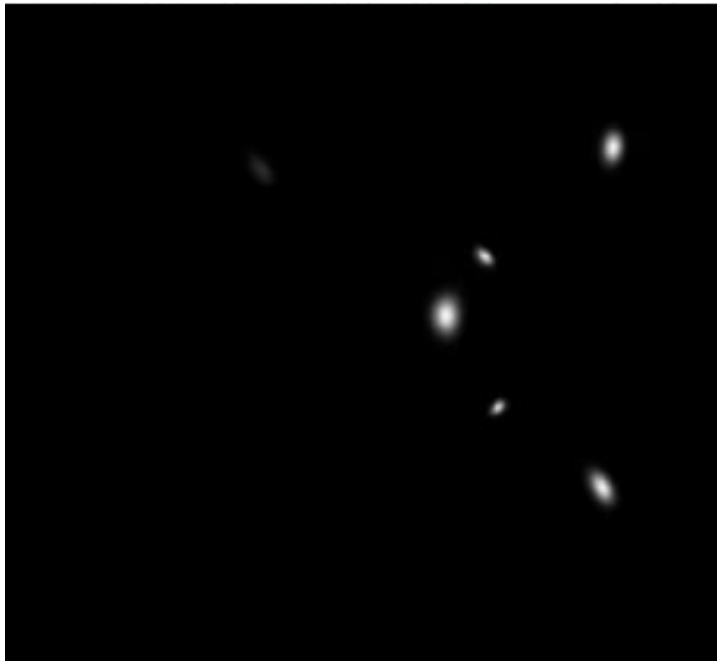
Phong shading

- **Specular reflection** is a reflection of light off a surface. It produces highlights, which can be observed on any shiny surface. Below, notice the white light reflecting off the apple.
- Specular reflection depends on the viewer position. When looking at an apple, If you move your head, you'll notice the specular highlight moves as well.
- In computer graphics, *shininess* is used to control the size of the highlight. Shinier objects have smaller highlights.
- A reflection of **s** about **n** produces the refection vector **r**. In the Phong model, the specular reflection is determined from the angle between **r** and **v**. This is also conveniently expressed as a dot product wit



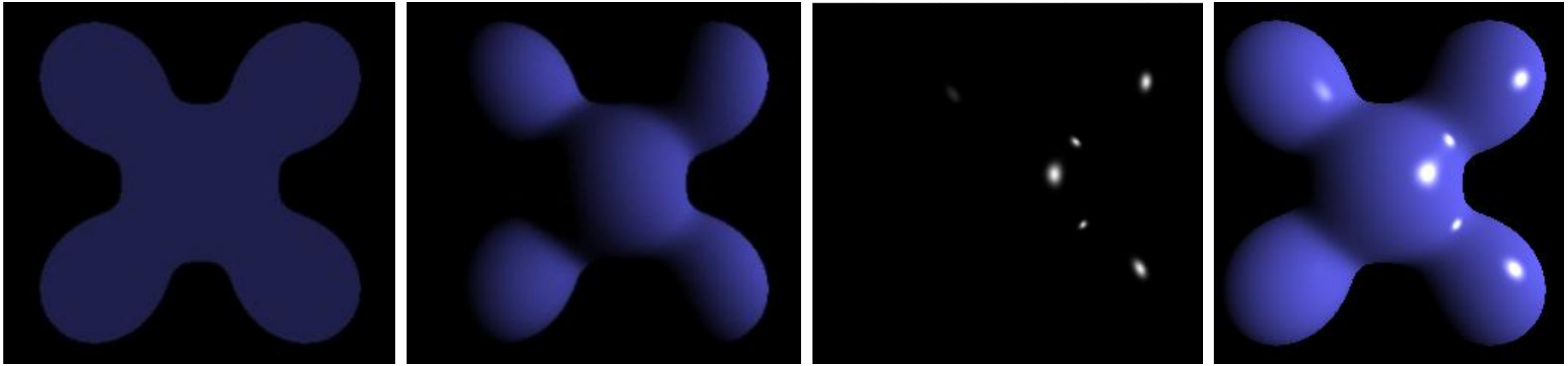
Phong model: specular term

- A light's specular component $\mathbf{l}_s$ is reflected by a surface. The amount of light reflection at each vertex is based on the reflection vector **r**, the viewing vector to the camera **v**, and the surface *specular material*, $\mathbf{m}_s$.
- The colour resulting from the specular reflection is $\mathbf{c}_s = (\mathbf{l}_s .* \mathbf{m}_s)\ (\mathbf{r} \bullet \mathbf{v})^n$, where n is the *shininess*.



- Some programmers only compute the specular term for points where $\mathbf{s} \bullet \mathbf{n} > 0$.

Full model: A + D + S



ambient + diffuse + specular = lit result

Phong reflectance model equation

$$\mathbf{c} = \mathbf{l}_a . * \mathbf{m}_a + \mathbf{l}_d . * \mathbf{m}_d \left[\max\left(\mathbf{s} \cdot \mathbf{n}, 0\right)\right] + \mathbf{l}_s . * \mathbf{m}_s \left[\max\left(\mathbf{r} \cdot \mathbf{v}, 0\right)\right]^n$$

where

c is the colour computed at the point

l_a, l_d, l_s are the light colours (ambient, diffuse, and specular)

$\mathbf{m}_a$, $\mathbf{m}_d$, $\mathbf{m}_s$ are the material reflectances (ambient, diffuse, and specular)

s is the normalised vector pointing to the light source

n is the unit normal

r is the normalised reflection vector

v is the normalised vector pointing to the camera

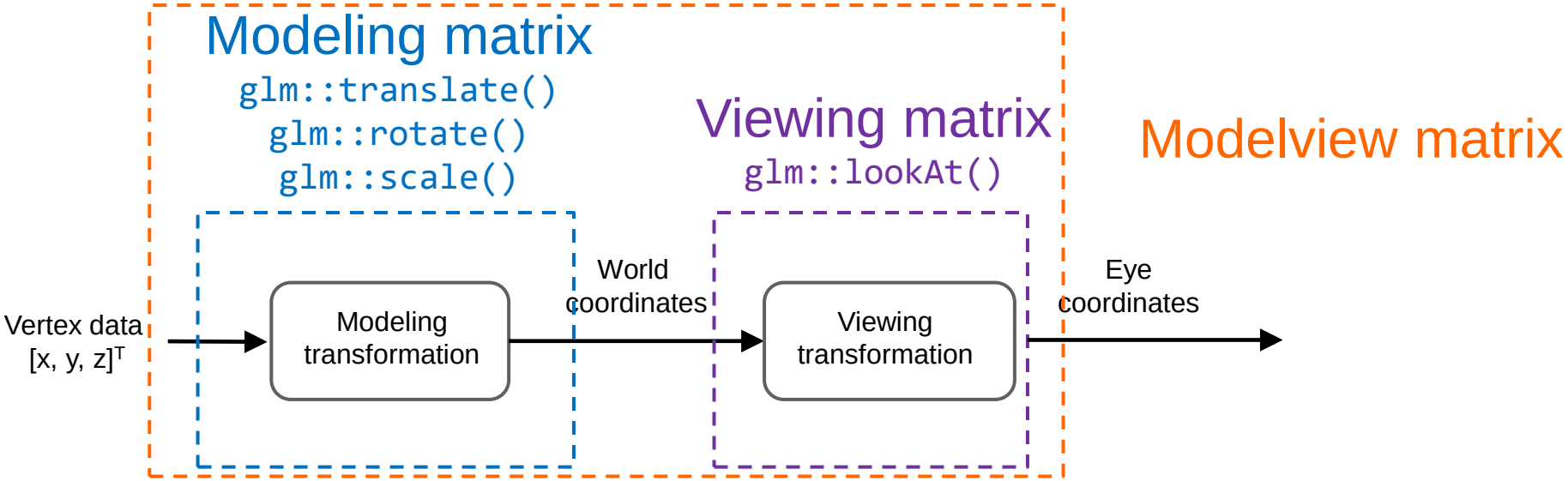
n is the shininess

Note: This is for a single light. If multiple lights are used, one computes a colour for each light source (and sums all the colours up).

Lighting done in eye coordinates

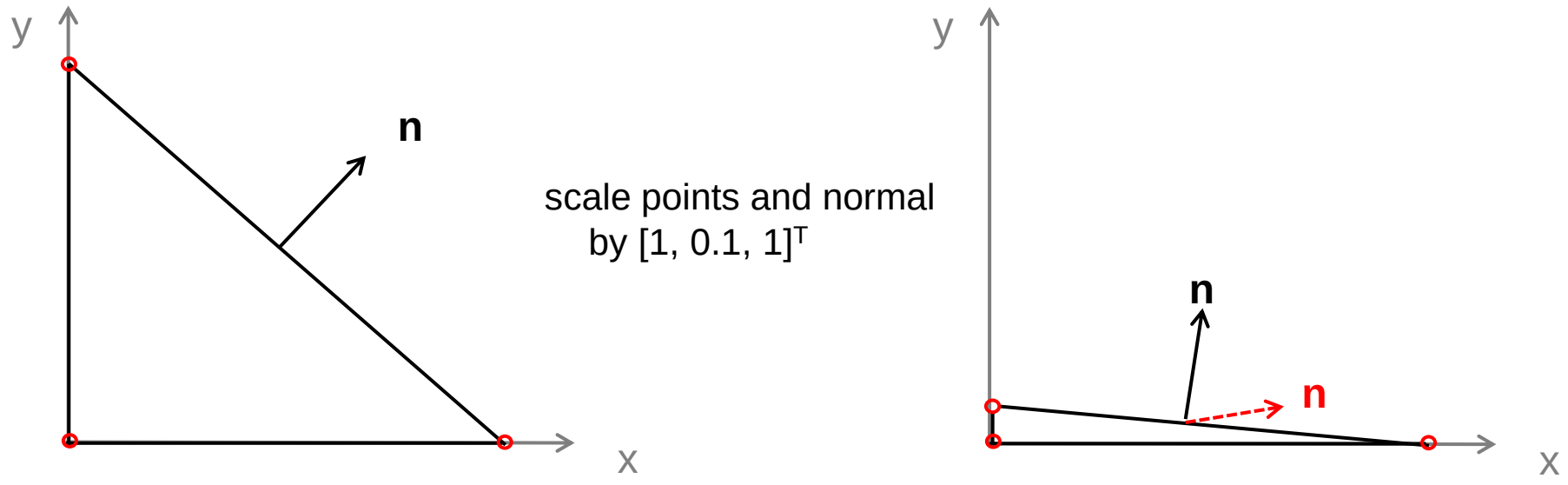
- Typically, all lighting computations are done in *eye coordinates*
  - Only one set of eye coordinates
  - Eye coordinates are computed as part of the camera model
  - Older versions of OpenGL provided light sources in eye coordinates
  - The vector **v** is easily computed in eye coordinates!
- Recall that to transform **points** to eye coordinates, all we have to do is multiply by the modelview matrix

```
vec4 vEyePosition = matrices.modelViewMatrix * vec4(inPosition, 1.0f);
```



Transforming normals

- However, we cannot just apply the modelview matrix to a **normal**, since we only want to change the normal's *orientation*.
- For example, consider the following case:



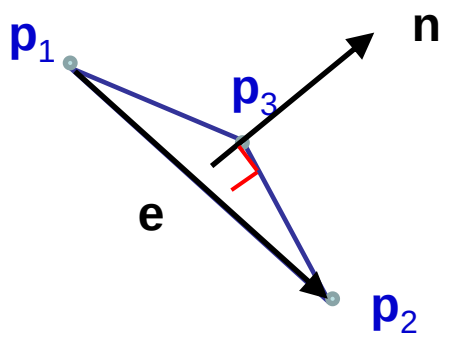
- We expect the normal to be rotating towards the y axis, not the x axis!

Normal matrix

- To transform a normal to eye coordinates, we use the *normal matrix*. It is also used to transform direction vectors (for example, a spotlight direction vector).
- The normal matrix **N** is the transpose of the inverse of the upper left 3x3 submatrix of the modelview matrix (denoted this as **M**). This part of the matrix represents rotation and scaling (translations don't affect the orientation of a normal vector)

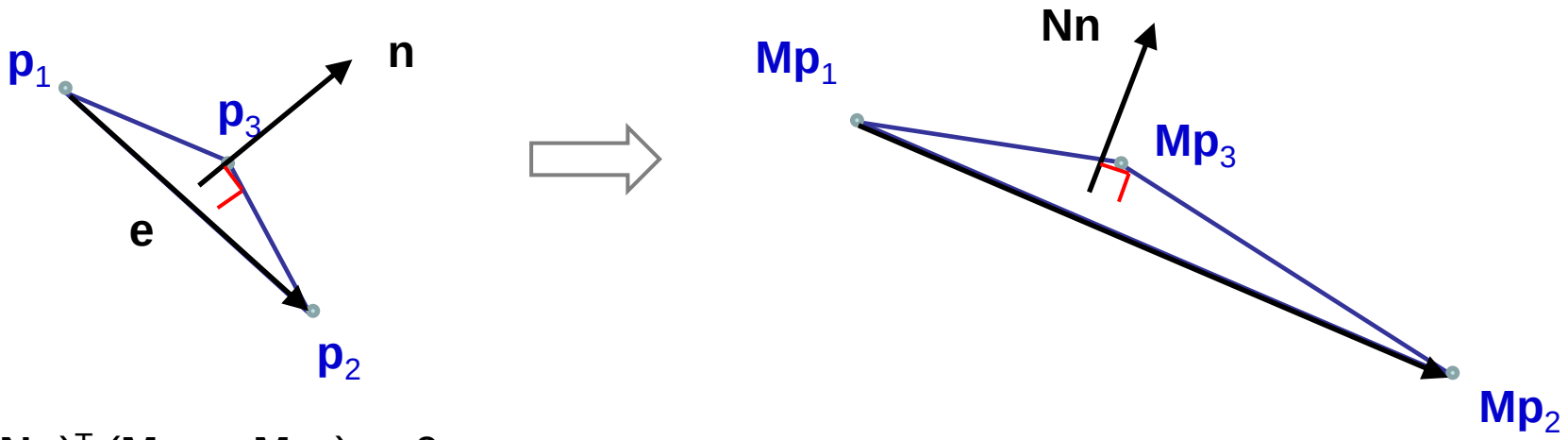
$$\mathbf{N} = (\mathbf{M}^{-1})^T$$

- Derivation:
  - Consider the triangle below. The normal **n** is orthogonal to *any* vector in the plane of the triangle, including its edges, for example edge $\mathbf{e} = \mathbf{p}_2 - \mathbf{p}_1$
  - That is, $\mathbf{n} \bullet \mathbf{e} = 0$. We did something just like this in the vector mathematics worksheet.
  - Note that the dot product can be written as a matrix multiplication, that is $\mathbf{n} \bullet \mathbf{e} = \mathbf{n}^T \mathbf{e} = 0$



Normal matrix

- We're going to apply the modelview matrix **M** to the points on the triangle.
- We'd like to find a transformation **N**, applied to the normal, so that the transformed normal is still orthogonal to the triangle after transformation.



- That is, $(\mathbf{Nn})^T (\mathbf{Mp}_2 - \mathbf{Mp}_1) = 0$
- This can be rewritten as $(\mathbf{Nn})^T \mathbf{M} (\mathbf{p}_2 - \mathbf{p}_1) = 0$
- Which can be rewritten as $\mathbf{n}^T \mathbf{N}^T \mathbf{M}\, \mathbf{e} = 0$
- We know $\mathbf{n}^T\mathbf{e} = 0$ from the previous slide. So if $\mathbf{N}^T \mathbf{M} = \mathbf{I}$, the equation will be satisfied. If we post-multiply each side by $\mathbf{M}^{-1}$, we get
- $\mathbf{N}^T = \mathbf{M}^{-1}$, and taking the transpose of both sides gives
- $\mathbf{N} = (\mathbf{M}^{-1})^T$

Computing the normal matrix

- The camera class computes the normal matrix using GLM operations:

```
// Compute normal matrix
glm::mat3 CCamera::ComputeNormalMatrix(const glm::mat4 &modelViewMatrix)
{
   return glm::transpose(glm::inverse(glm::mat3(modelViewMatrix)));
}
```

⇨ If the modelview matrix changes, we must recompute the normal matrix.

```
// Render the horse
modelViewMatrixStack.Push();
   modelViewMatrixStack.Translate(glm::vec3(0.0f, 0.0f, 0.0f));
   modelViewMatrixStack.Rotate(glm::vec3(0.0f, 1.0f, 0.0f), 180.0f);
   modelViewMatrixStack.Scale(2.5f);
   m_pMainProgram->SetUniform("matrices.modelViewMatrix",
                              modelViewMatrixStack.Top());
   m_pMainProgram->SetUniform("matrices.normalMatrix",
                 m_pCamera->ComputeNormalMatrix(modelViewMatrixStack.Top()));
   m_pObjHorse->Render();
modelViewMatrixStack.Pop();
```

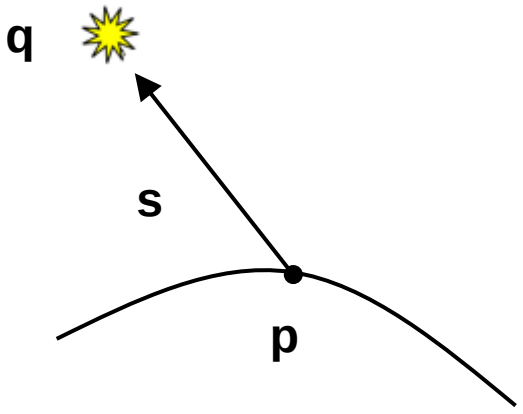
Normal transformation

- Therefore, we will apply the normal matrix to transform vertex normals into eye coordinates.
- This is typically done in the vertex shader.
- Be sure to renormalize after the transformation.

```
n = normalize(matrices.normalMatrix * inNormal);
```

Vector to light source

- Consider a point light. The light has a position in world coordinates, and converted to eye coordinates using the view matrix, forming a point **q**.
  - Note that in the template code, the view matrix is first placed on the matrix stack (using the call `lookAt()`) and then the light position is transformed to eye coordinates.
- In the vertex shader, the vertex position is read from a VBO and converted to eye coordinates using the modelview matrix, forming a point **p**.

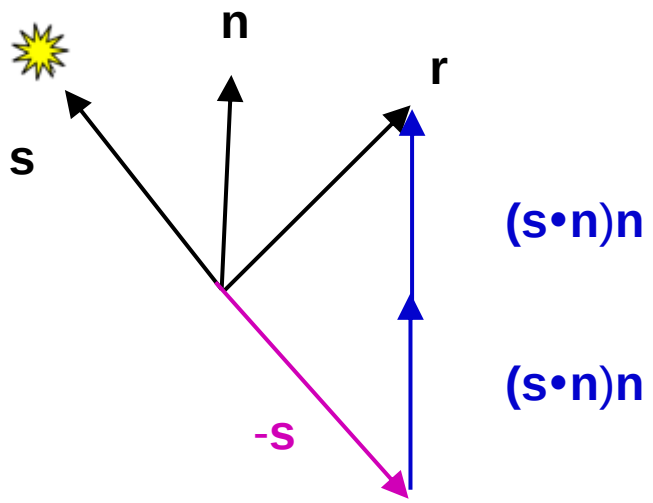



```
s = normalize(q – p);
```

Reflection vector

- Assuming **s** and **n** are normalised, the reflection vector can be computed as

$$\mathbf{r}=(-\mathbf{s}+2(\mathbf{s}\bullet\mathbf{n})\mathbf{n})$$



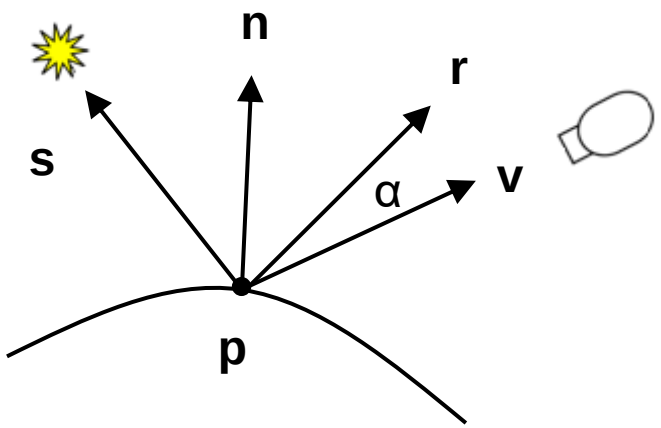
- In GLM or GLSL, this can be achieved by calling `r = reflect(-s, n);`

Viewing vector

- Question: In eye coordinates, where is the camera?
- Answer: At the origin. Eye coordinates are based on the camera.
- Therefore, **v** is really simple!

$$\mathbf{v} = -\mathbf{p}$$

Note the negative sign




```
vec3 v = normalize(-p);  if p is a vec3
vec3 v = normalize(-p.xyz); if p is a vec4
```

Implementing the Phong model

- The template code implements a (per-vertex) Phong reflectance model for a single light.
- In the vertex shader, there are data structures for light and materials. The values are passed in from the client program.

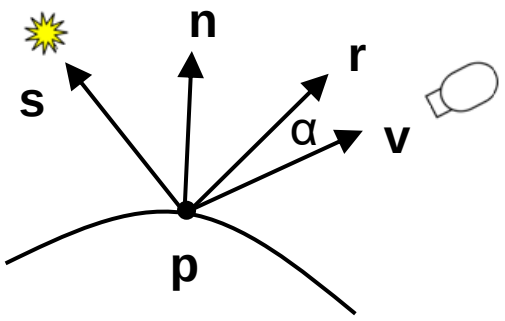
```
// Structure holding light information
struct LightInfo
{
   vec4 position;
   vec3 La;
   vec3 Ld;
   vec3 Ls;
};
// Structure holding material information
struct MaterialInfo
{
   vec3 Ma;
   vec3 Md;
   vec3 Ms;
   float shininess;
};
// Lights and materials passed in as uniform variables from client program
uniform LightInfo light1;
uniform MaterialInfo material1;
```

Implementing the Phong model

- In the vertex shader, a function PhongModel implements the equation

```
vec3 PhongModel(vec4 p, vec3 n)
{
   vec3 s = normalize(vec3(light1.position - p));
   vec3 v = normalize(-p.xyz);
   vec3 r = reflect(-s, n);
   vec3 ambient = light1.La * material1.Ma;
   float sDotN = max(dot(s, n), 0.0f);
   vec3 diffuse = light1.Ld * material1.Md * sDotN;
   vec3 specular = vec3(0.0f);
   if (sDotN > 0.0f)
       specular = light1.Ls * material1.Ms * pow(max(dot(r, v), 0.0f),
                                                 material1.shininess);

   return ambient + diffuse + specular;
}
```



- In the fragment shader, the colour is interpolated across the primitive and used to set a fragment's colour

Implementing the Phong model

- On the client side, we must provide the following to the shaders:
  - The light properties
  - The material properties
  - The projection, modelview, and normal matrices
- The projection matrix typically does not change when rendering a frame. Therefore, it can be sent once

```
pMainProgram->SetUniform("matrices.projMatrix",
                         pCamera->GetPerspectiveProjectionMatrix());
```

- However, the modelview matrix (and therefore the normal matrix) typically change for each object being rendered. Therefore we must update these before rendering the objects.

Tip

⇨ It is best to compute the normal matrix on the client side and pass it into the shaders.

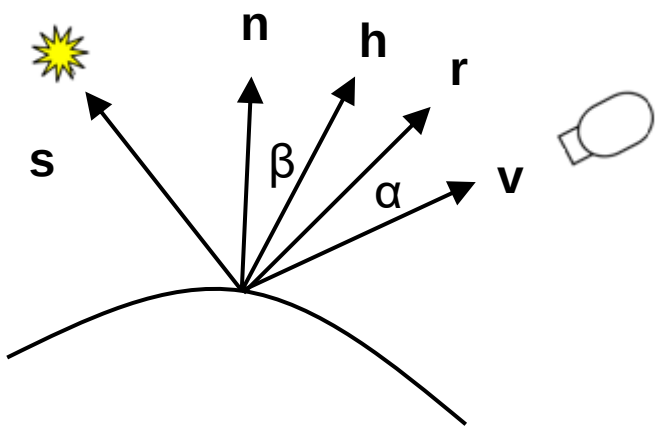
- While it is possible to write GLSL code to compute the normal matrix in a shader, the vertex / fragment shaders often execute many times when rendering an object.
- Matrix inversion is computationally expensive [standard algorithms run in $O(N^3)$ for a NxN matrix].
- The normal matrix does not change when rendering a single object. It is usually more efficient to compute it on the client side and pass it in through a uniform variable.

Blinn-Phong model: specular term

- For computational efficiency, Blinn modified the Phong model.
- Instead of basing the specular reflection on the angle *α* between **r** and **v**, it is based on the angle *β* between **h** and **n**. Here, **h** is the *halfway vector*, which is halfway between **s** and **v**.

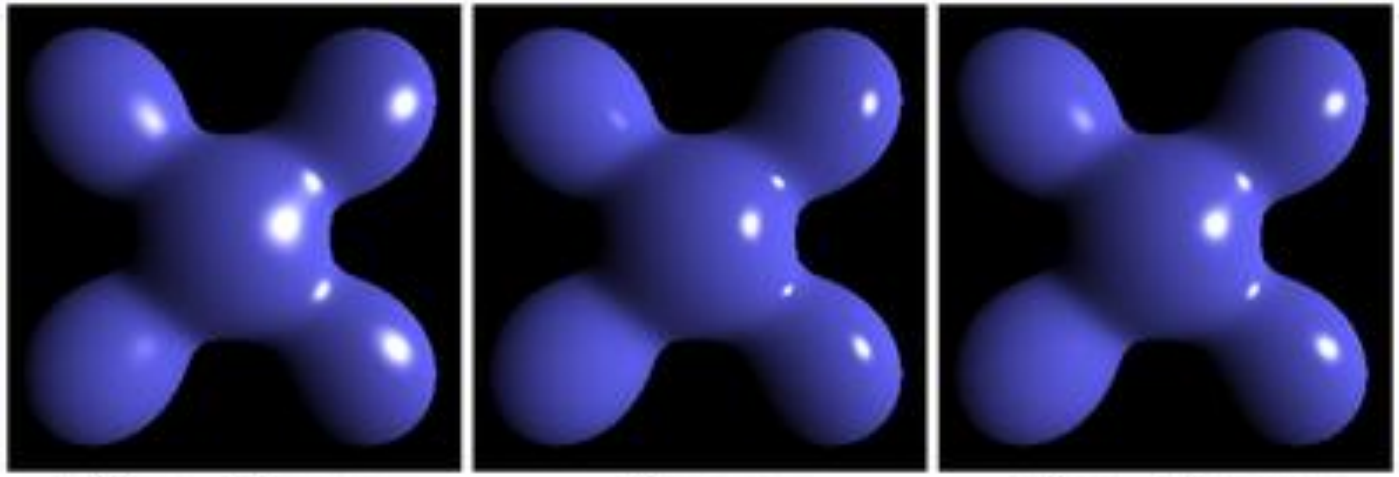
$$\mathbf{h} = \frac{\mathbf{s} + \mathbf{v}}{||\mathbf{s} + \mathbf{v}||}$$

- **h** is slightly faster to compute than **r**. Note though, *β* is a different angle than *α*. Using a different angle affects the specular reflection. However, the shininess can be modified to provide a similar effect as Phong shading.



Blinn-Phong model: specular term

- A light's specular component $\mathbf{l}_s$ is reflected by a surface. The amount of light reflection at each vertex is based on the halfway vector **h**, the surface normal **n**, and the surface *specular material*, $\mathbf{m}_s$.
- The colour resulting from the specular reflection is $\mathbf{c}_s = (\mathbf{l}_s .* \mathbf{m}_s)\ (\mathbf{h} \bullet \mathbf{n})^n$, where n is the shininess.



Blinn–Phong | Phong | Blinn–Phong (higher exponent)

Blinn-Phong reflectance model equation

$$\mathbf{c} = \mathbf{l}_a . * \mathbf{m}_a + \mathbf{l}_d . * \mathbf{m}_d \left[\max\left(\mathbf{s} \cdot \mathbf{n}, 0\right)\right] + \mathbf{l}_s . * \mathbf{m}_s \left[\max\left(\mathbf{h} \cdot \mathbf{n}, 0\right)\right]^n$$

Different to previous equation

where

c is the colour computed at the point

l_a, l_d, l_s are the light colours (ambient, diffuse, and specular)

$\mathbf{m}_a$, $\mathbf{m}_d$, $\mathbf{m}_s$ are the material reflectances (ambient, diffuse, and specular)

s is the normalised vector pointing to the light source

n is the unit normal

h is the halfway vector

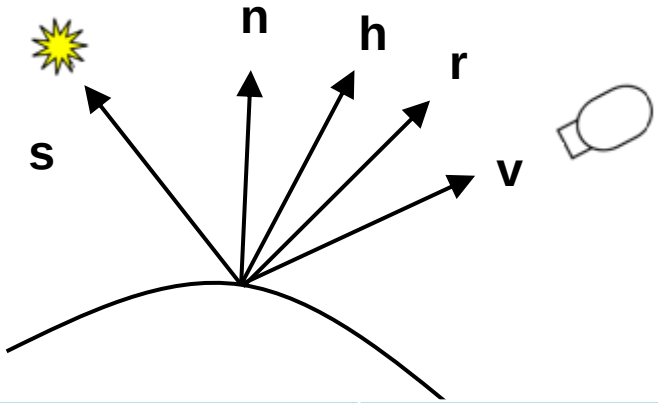
n is the shininess

Note: This is for a single light. If multiple lights are used, one computes a colour for each light source (and sums all the colours up).

Exercise

- Fill out the following table for the terms in the Blinn-Phong reflectance model
- Hint: remember the equation

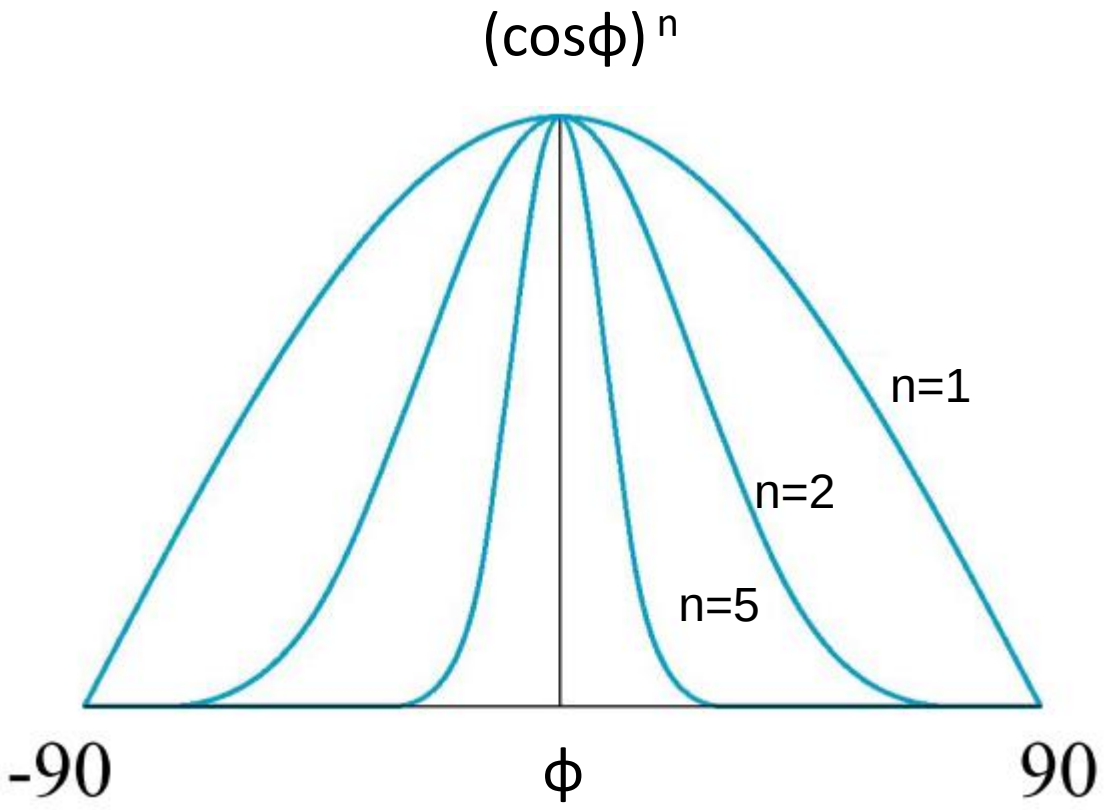
$$\mathbf{c} = \mathbf{l}_a. * \mathbf{m}_a + \mathbf{l}_d. * \mathbf{m}_d \left[\max\left(\mathbf{s} \cdot \mathbf{n}, 0\right)\right] + \mathbf{l}_s. * \mathbf{m}_s \left[\max\left(\mathbf{h} \cdot \mathbf{n}, 0\right)\right]^n$$



	Ambient	Diffuse	Specular
Varies based on vertex normal			
Varies based on camera position			
Varies based on light source position			

Shininess

$\cos \phi = \mathbf{h} \bullet \mathbf{n}$ or $\mathbf{r} \bullet \mathbf{v}$



Shininess

- Shininess values typically range from 1 to 128. Higher values result in more focussed specular highlights.

5 50 128

Shininess (using the Blinn-Phong model)

The limitations of a local lighting model

ambient diffuse specular lit result

- Notice the ear is brightly lit despite being in shadow. This is a consequence of the OpenGL local lighting model approximation.

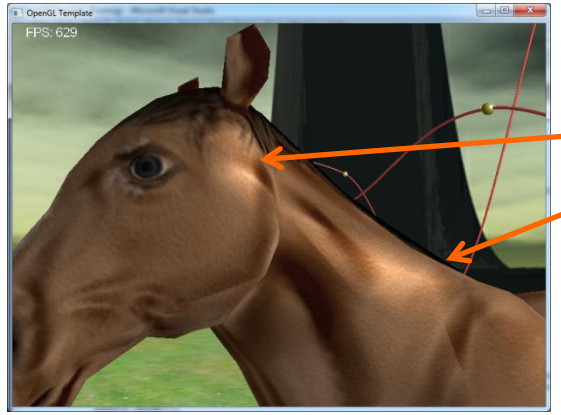
What about textured surfaces?

- One can evaluate the lighting equation, then combine with the colour supplied by a texture in the fragment shader

```
vec3 vColour = PhongModel(p, n);

// Get the texel colour from the texture sampler
vec4 vTexColour = texture(sampler0, vTexCoord);

// Combine object colour and texture
vec4 vMixedColour = vTexColour * vec4(vColour, 1.0f);
```



specular highlights

Code walkthrough

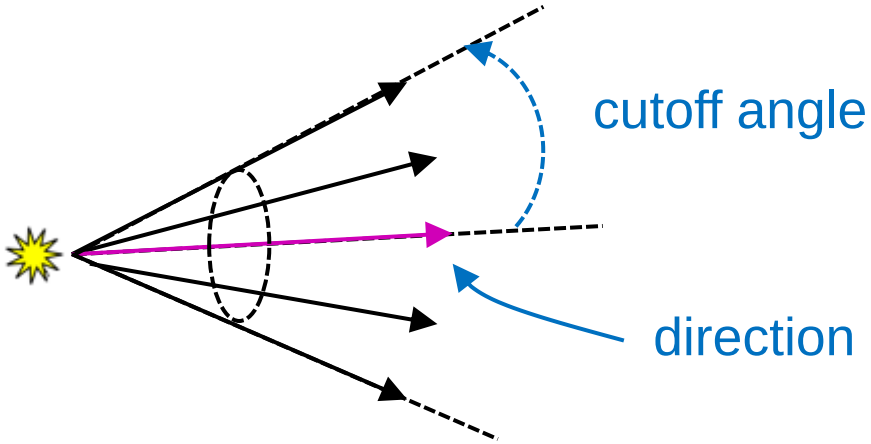
- Let’s take a look at how lighting is implemented in the template code.

Directional light source

- A directional light source is one where the rays are parallel.
- This is a good model for light coming from distance sources (like the sun).
- In the implementation, a direction vector to the light source is given.
- Then, one uses the normal matrix to convert the direction vector to eye coordinates to form **s**.

Spotlight

- A spotlight radiates light within a cone. In addition, light is modeled so that it is brightest along the direction the spotlight is pointing.
- For this we required three additional light properties:
  - a *direction* the spotlight is pointing
  - a *cutoff* angle, used to define a cone around the direction vector
  - an *exponent*, describing how light falls off from the centre



Spotlight

- When rendering a point, we first check to see if it's inside the cone. If it is not inside the cone, we just return the ambient term.
- If the point is inside the cone, we determine the intensity of the spotlight at the point based on the form

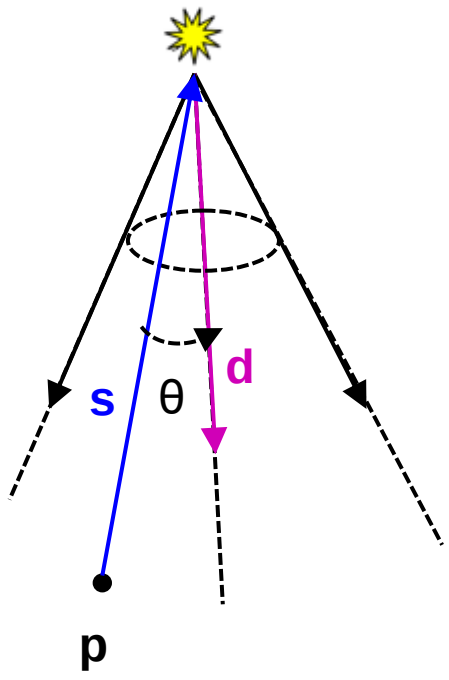
$$f = ((-\mathbf{s}) \bullet \mathbf{d})^a$$

where

s is the vector to the light source

d is the spotlight direction (in eye coordinates)

a is an exponential falloff term



Spotlight example

```
vec3 BlinnPhongSpotlightModel(vec4 p, vec3 n)
{
   vec3 s = normalize(vec3(light1.position - p));
   float angle = acos(dot(-s, light1.direction));
   float cutoff = radians(clamp(light1.cutoff, 0.0, 90.0));
   vec3 ambient = light1.La * material1.Ma;
   if (angle < cutoff) {
      float spotFactor = pow(dot(-s, light1.direction), light1.exponent);
      vec3 v = normalize(-eyePosition.xyz);
      vec3 h = normalize(v + s);
      n = normalise(n);
      float sDotN = max(dot(s, n), 0.0);
      vec3 diffuse = light1.Ld * material1.Md * sDotN;
      vec3 specular = vec3(0.0);
      if (sDotN > 0.0)
         specular = light1.Ls * material1.Ms * pow(max(dot(h, n), 0.0),
                    material1.shininess);
      return ambient + spotFactor * (diffuse + specular);
    } else
       return ambient;
}
```

Attenuation

- It is also possible to model attenuation of the light with distance from the light source. Points farther from the light source receive less light.
- Attenuation factor:

$$\frac{1}{k_c + k_l d + k_q d^2}$$

where

d = distance between the light's position and the vertex
k_c = constant attenuation constant
k_l = linear attenuation constant
k_q = quadratic attenuation constant

- This would be applied to the **diffuse** and **specular** terms

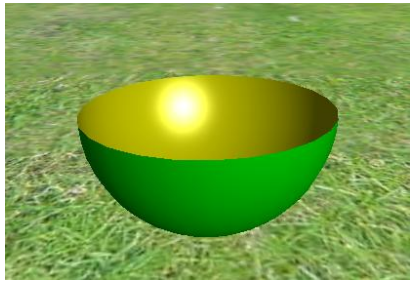
⇨ Think about the distances in your world to create appropriate constants

http://www.paulsprojects.net/opengl/ppatten/ppatten.html

Two-sided shading

- It is possible to shade the back side of a triangle differently from the front side. Recall that the front and back sides are specified by the triangle winding discussed in Lecture 1.
- The built-in fragment shader variable `gl_FrontFacing` will indicate if the fragment is front or back facing.
- Example: Rendering a hemisphere's front faces green, and swizzling on the back faces to change their colour (to yellow):

```
if (gl_FrontFacing)
   vColour = PhongModel(p, normalize(n));
else {
   vColour = PhongModel(p, normalize(-n));
   vColour = vColour.ggb;
}
```



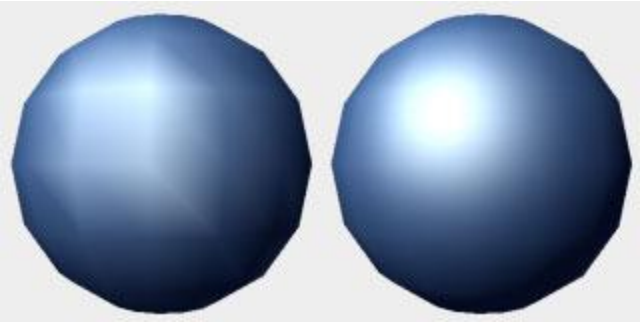
⇨ If rendering back faces, be sure `GL_CULL_FACE` is disabled!

Multiple lights and clamping

- For a scene with multiple lights, one loops over all the light sources, and computes a colour for each light
- The resulting colour will be the sum
- Note that colours are *clamped* to the range [0, 1]
  - When passed from the vertex shader to the fragment shader.
  - Before being written to the colour buffer.

Per-vertex vs. per-fragment lighting

- The reflectance model can be computed in the vertex or the fragment shader.
- When it is computed in the vertex shader, one is said to be implementing *per-vertex lighting*.
- When it is computed in the fragment shader, one is said to be implementing *per-fragment lighting*.
- For many scenes, there are fewer vertices than fragments to be processed. Therefore, per-vertex lighting is typically faster.
- However, in per-vertex lighting, the lighting is only evaluated at the vertices and interpolated along the primitive. In this case, the tessellation of the surface affects the quality of the shading.



Per-vertex lighting | Per-fragment lighting